

ACTIVIDAD IX

Ingeniería Orientada a Objetos

Taller Docker — OO Clínica Web en Contenedor

CRUD de Pacientes y Citas con Django,
PostgreSQL y Podman

Universidad Surcolombiana

Facultad de Ingeniería — Programa de Ingeniería de Software
Neiva — Huila

ESTUDIANTES

Jorge León Chavarro Alarcón Código: 20231211354

Juan David Rivera Chaté Código: 20231213382

Luis Alejandro Restrepo Medina Código: 20231211869

DOCENTE

Eduardo Martínez Vidal

26 de abril
de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Justificación técnica	2
3.1. Uso de Podman como runtime de contenedores	2
3.2. Entorno de desarrollo	3
4. Descripción de la Aplicación	4
4.1. Dominio	4
4.2. Paradigma orientado a objetos	4
4.3. Stack tecnológico	6
4.4. Interfaz del CRUD	6
5. Arquitectura de Contenedores	8
5.1. Servicio web	9
5.2. Servicio db	9
5.3. Persistencia	9
5.4. Verificación de la red privada	9
6. Archivos clave del proyecto	10
6.1. Dockerfile	10
6.2. compose.yml	11
6.3. entrypoint.sh	12
6.4. Variables de entorno	13
7. Proceso de despliegue	13
7.1. Requisitos previos	13
7.2. Pasos de despliegue	13
7.3. Evidencias del despliegue	14
7.4. Credenciales del administrador (entorno de demostración)	14
8. Verificación de la persistencia	15
9. Publicación en Docker Hub	16
9.1. Autenticación	16
9.2. Etiquetado	16
9.3. Publicación	16
10. Conclusiones	17
10.1. Cumplimiento de objetivos	17
10.2. Lecciones técnicas	17
11. Referencias	18

1 Introducción

El presente informe documenta el desarrollo de la **Actividad IX** de la asignatura Ingeniería de Software Orientada a Objetos, cuyo objetivo es construir una aplicación con paradigma orientado a objetos, conectada a una base de datos relacional, empaquetada en un contenedor con persistencia de datos mediante volúmenes, y finalmente publicada en Docker Hub.

Se optó por desarrollar una aplicación web de gestión de pacientes y citas para una clínica, implementada en Python con el framework **Django 6.0.4**, respaldada por **PostgreSQL 16** como motor de base de datos. Todo el entorno se orquesta mediante **Podman** (alternativa libre y compatible con Docker, autorizada por el docente) en lugar de Docker, aprovechando que el equipo de trabajo cuenta con sistemas operativos basados en Linux sin la necesidad de un demonio privilegiado.

2 Objetivos

2.1 Objetivo general

Desarrollar una aplicación orientada a objetos, en contenedor, con persistencia de datos mediante volúmenes y publicada en Docker Hub.

2.2 Objetivos específicos

- Implementar un CRUD relacional con al menos dos entidades siguiendo el paradigma orientado a objetos.
- Diseñar una arquitectura multi-contenedor con separación de responsabilidades entre aplicación y base de datos.
- Garantizar la persistencia de la información mediante el uso de volúmenes nombrados.
- Automatizar el arranque completo del sistema mediante `podman-compose` y un script de inicio.
- Publicar la imagen resultante en Docker Hub para su distribución.

3 Justificación técnica

3.1 Uso de Podman como runtime de contenedores

La consigna de la actividad establece Docker como herramienta principal. El equipo solicitó al docente autorización explícita para utilizar **Podman** como alternativa, permiso que fue concedido para esta entrega. La tabla siguiente sintetiza las diferencias técnicas relevantes que motivaron la elección.

Característica	Docker	Podman
Arquitectura	Cliente y demonio <code>dockerd</code> como root	Sin demonio; procesos del propio usuario
Modelo de seguridad	Requiere grupo <code>docker</code> (privilegios de root)	<i>Rootless</i> por defecto
Compatibilidad	Formato OCI	Formato OCI (compatible con imágenes de Docker)
Orquestación local	<code>docker compose</code>	<code>podman-compose</code>
Registries	Cualquier registry OCI	Cualquier registry OCI, incluido Docker Hub

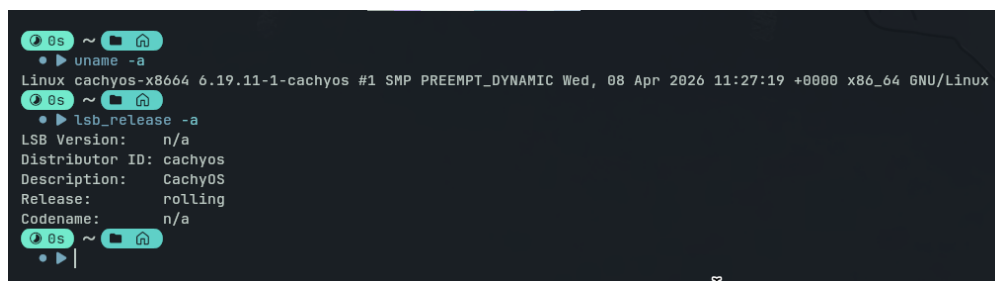
Cuadro 1: Comparación técnica entre Docker y Podman

Razones concretas de la elección:

- **Compatibilidad bidireccional:** el `Dockerfile` y el `compose.yml` son 100 % compatibles con Docker. La imagen publicada puede ejecutarse con `docker compose up` sin modificar un solo archivo.
- **Modelo *rootless*:** los contenedores se ejecutan como procesos del usuario, sin necesidad de un demonio privilegiado en segundo plano.
- **Instalación simplificada en Arch Linux:** `podman` y `podman-compose` están disponibles en los repositorios oficiales y se instalan con un solo comando `pacman`.

3.2 Entorno de desarrollo

El proyecto se desarrolló sobre **Linux nativo** (CachyOS, distribución derivada de Arch Linux), instalado directamente sobre la máquina física. Esto corresponde a la opción de mayor exigencia entre las tres planteadas en la guía del docente (Básica, Media y Alta).



```

0s ~ 
• ▶ uname -a
Linux cachyos-x86_64 6.19.11-1-cachyos #1 SMP PREEMPT_DYNAMIC Wed, 08 Apr 2026 11:27:19 +0000 x86_64 GNU/Linux
0s ~ 
• ▶ lsb_release -a
LSB Version:    n/a
Distributor ID: cachyos
Description:    CachyOS
Release:        rolling
Codename:       n/a
0s ~ 
• ▶

```

Figura 1: Sistema operativo del entorno de desarrollo: CachyOS (Linux nativo).

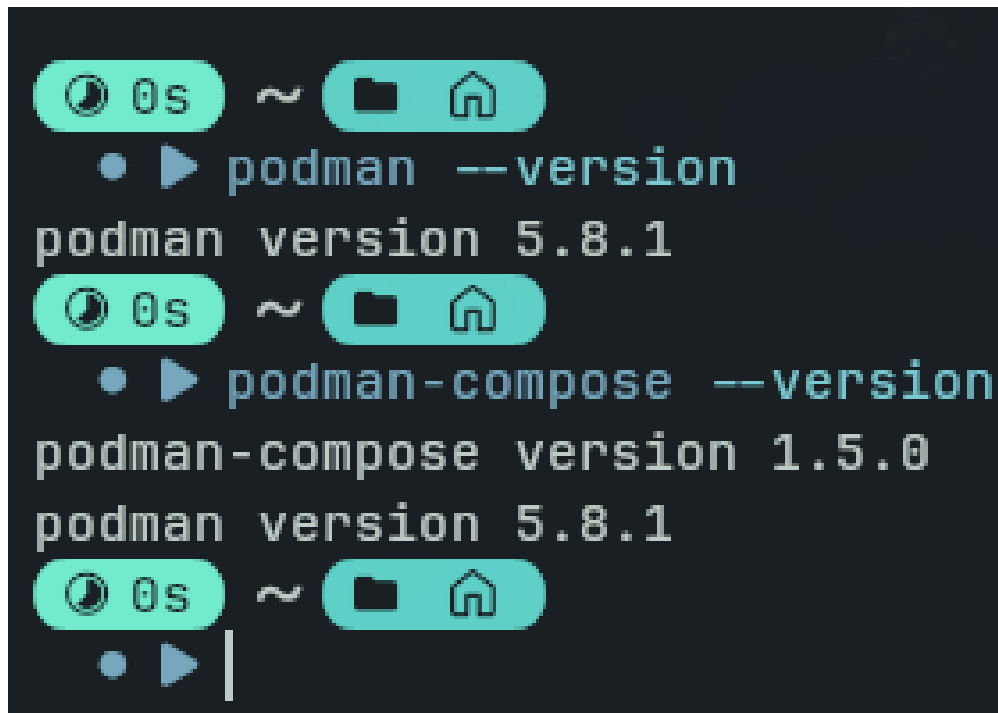
A terminal window with a dark background and light blue text. The prompt is '~' with a battery icon and '0s'. The first command is 'podman --version', which outputs 'podman version 5.8.1'. The second command is 'podman-compose --version', which outputs 'podman-compose version 1.5.0' followed by 'podman version 5.8.1' on the next line. The prompt is '~' with a battery icon and '0s'.

Figura 2: Versiones de Podman y podman-compose instaladas en el entorno.

4 Descripción de la Aplicación

4.1 Dominio

La aplicación modela la gestión básica de una clínica. Permite registrar pacientes con sus datos de contacto y agendar citas asociadas a cada paciente con fecha, hora y motivo. Se implementó un CRUD completo (crear, listar, editar, eliminar) para ambas entidades.

4.2 Paradigma orientado a objetos

Django implementa una variante del patrón *Active Record* en su ORM, con elementos de *Data Mapper* en sus `QuerySet`: cada tabla de la base de datos se representa como una clase Python que hereda de `django.db.models.Model` y expone atributos tipados y métodos propios del dominio. A continuación se muestran las dos clases principales.

```
1 from django.db import models
2 from django.core.validators import RegexValidator
3
4
5 class Paciente(models.Model):
6     nombre = models.CharField(max_length=150)
7     cedula = models.CharField(
8         max_length=20,
9         unique=True,
10        validators=[
11            RegexValidator(
12                regex=r"^\d+$",
13                message="La cedula debe contener solo numeros.",
```

```
14         )
15     ],
16 )
17     telefono = models.CharField(
18         max_length=20,
19         validators=[
20             RegexValidator(
21                 regex=r"^\+?\d+$",
22                 message="El telefono debe contener solo numeros.",
23             )
24         ],
25     )
26     email = models.EmailField()
27
28     def __str__(self):
29         return f"{self.nombre} ({self.cedula})"
30
31
32 class Cita(models.Model):
33     paciente = models.ForeignKey(Paciente, on_delete=models.CASCADE)
34     fecha = models.DateTimeField()
35     motivo = models.TextField()
36
37     def __str__(self):
38         return f"{self.paciente.nombre} - {self.fecha:%Y-%m-%d %H:%M}"
```

Listing 1: clinica/models.py

Puntos clave del diseño OO:

- Cada entidad es una **clase** con atributos encapsulados y tipados.
- La clase **Paciente** incorpora **validadores** reutilizables (**RegexValidator**) que garantizan la integridad del dato a nivel de clase, no únicamente a nivel de formulario.
- La relación **1 a N** entre **Paciente** y **Cita** se expresa mediante una **ForeignKey** con política de borrado en cascada.
- El método mágico **__str__** ofrece una representación en texto útil para depuración, logs y vistas administrativas.
- El CRUD se implementa mediante *Class-Based Views* (**ListView**, **CreateView**, **UpdateView**, **DeleteView**) que heredan de clases base de Django, reutilizando lógica y promoviendo el principio de inversión de dependencias.

4.3 Stack tecnológico

Componente	Tecnología
Lenguaje	Python 3.12
Framework web	Django 6.0.4
Servidor de aplicación	Gunicorn 23
Servidor de estáticos	WhiteNoise 6.9
Base de datos	PostgreSQL 16 (alpine)
Driver DB	psycopg2-binary 2.9.11
Gestión de entorno	django-environ 0.13
Contenedores	Podman 5.8 + podman-compose 1.5
Registro de imágenes	Docker Hub

Cuadro 2: Stack tecnológico del proyecto

4.4 Interfaz del CRUD

A continuación se muestra la interfaz funcional sobre la cual el usuario opera. Las vistas implementan las cuatro operaciones del CRUD para las dos entidades del dominio (Paciente y Cita).

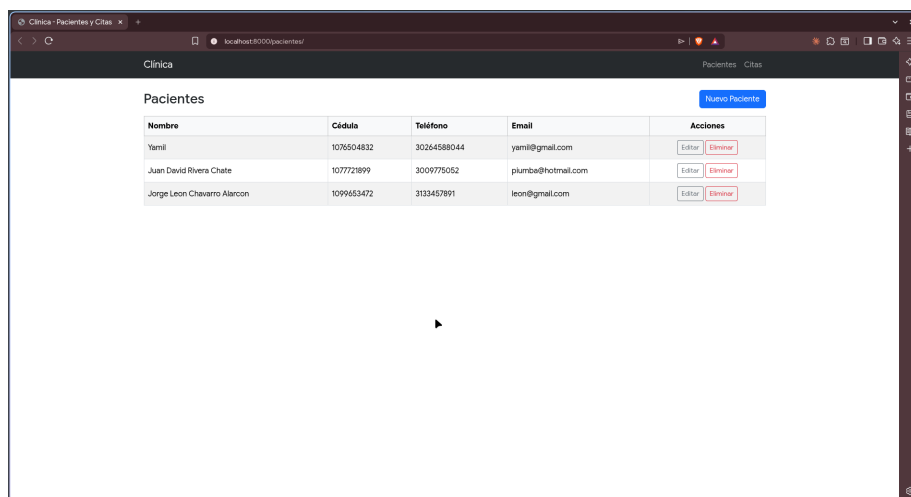
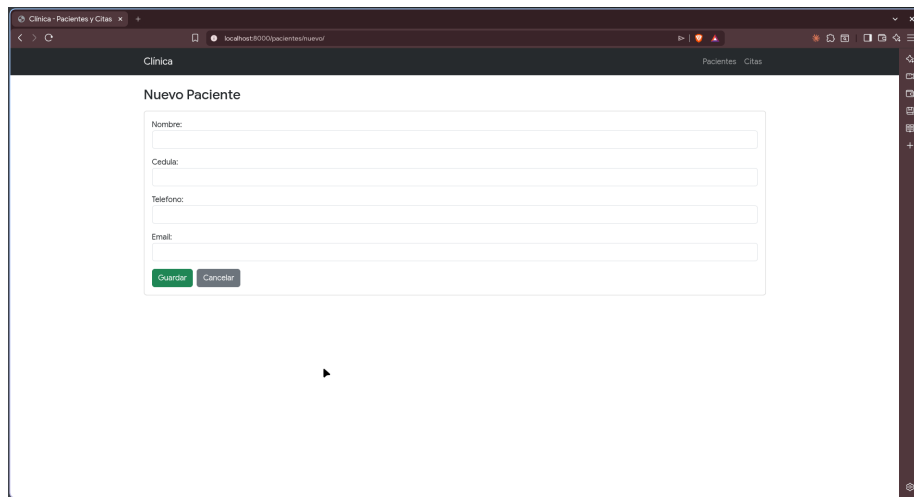
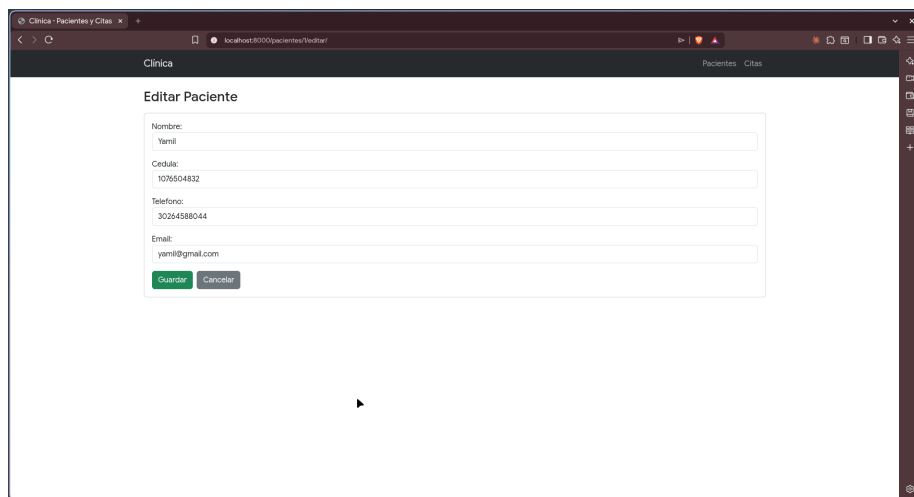


Figura 3: Listado de pacientes registrados.



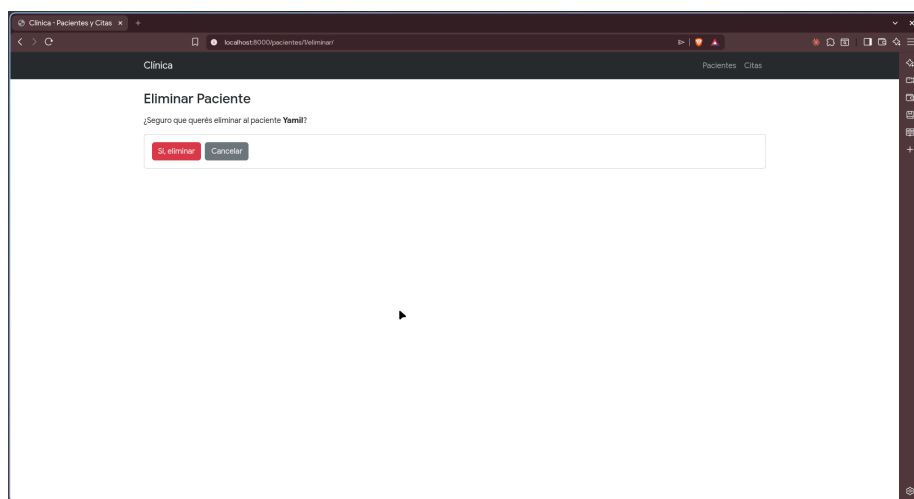
The screenshot shows a web browser window with the address bar displaying 'localhost:5000/pacientes/nuevo/'. The page title is 'Clínica' and the breadcrumb is 'Pacientes > Citas'. The main heading is 'Nuevo Paciente'. Below it is a form with four input fields: 'Nombre:', 'Cédula:', 'Teléfono:', and 'Email:'. At the bottom of the form are two buttons: 'Guardar' (green) and 'Cancelar' (grey).

Figura 4: Formulario de creación de paciente.



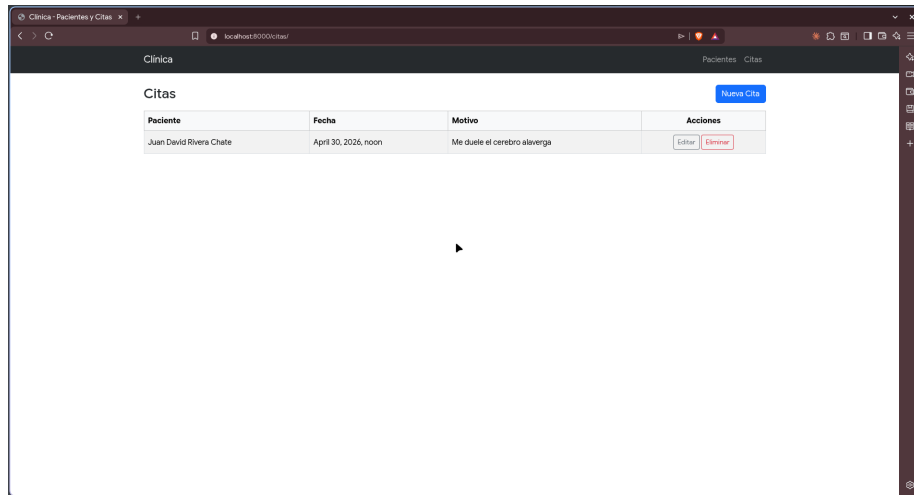
The screenshot shows a web browser window with the address bar displaying 'localhost:5000/pacientes/Editar/'. The page title is 'Clínica' and the breadcrumb is 'Pacientes > Citas'. The main heading is 'Editar Paciente'. Below it is a form with five input fields, each containing pre-filled data: 'Nombre:' (Yamil), 'Cédula:' (1076504832), 'Teléfono:' (30264588044), 'Email:' (yamil@gmail.com), and a fifth empty field. At the bottom of the form are two buttons: 'Guardar' (green) and 'Cancelar' (grey).

Figura 5: Formulario de edición de paciente.



The screenshot shows a web browser window with the address bar displaying 'localhost:5000/pacientes/eliminar/'. The page title is 'Clínica' and the breadcrumb is 'Pacientes > Citas'. The main heading is 'Eliminar Paciente'. Below it is a confirmation message: '¿Seguro que quieres eliminar al paciente Yamil?'. At the bottom of the message are two buttons: 'Si, eliminar' (red) and 'Cancelar' (grey).

Figura 6: Confirmación de eliminación de paciente.



Paciente	Fecha	Motivo	Acciones
Juan David Rivera Chate	April 30, 2026, noon	Me duele el cerebro silberga	Editar Eliminar

Figura 7: Listado de citas asociadas a pacientes (relación 1 a N mediante `ForeignKey`).

5 Arquitectura de Contenedores

La solución se compone de **dos contenedores** orquestados por `podman-compose`, un **volumen nombrado** para la persistencia de la base de datos y una **red privada** para aislar la comunicación entre servicios.

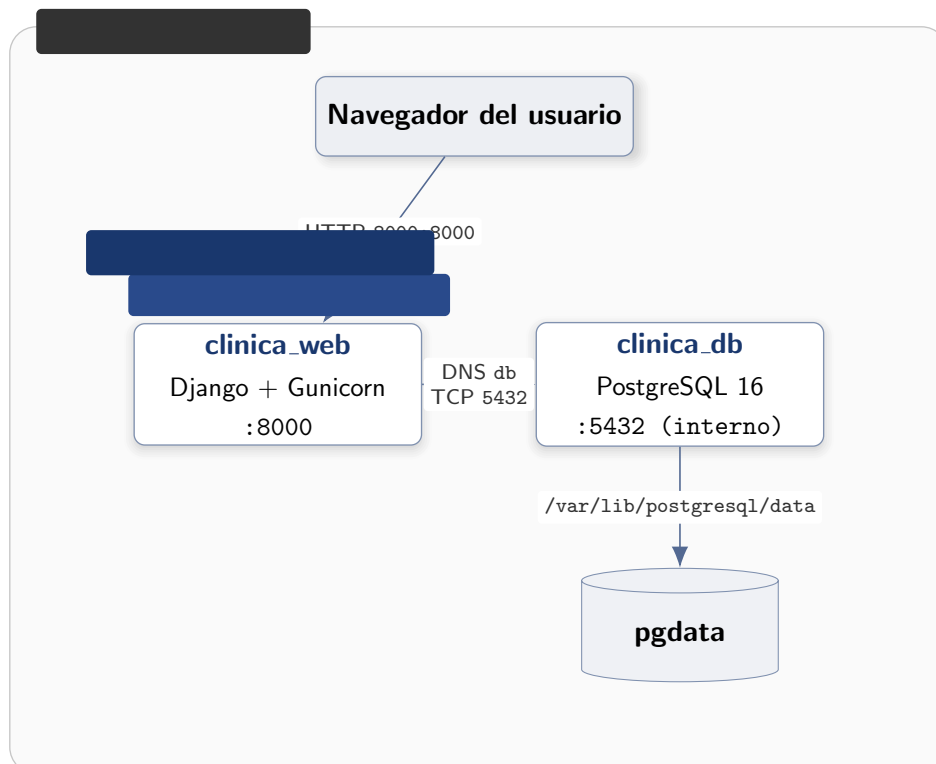


Figura 8: Arquitectura de contenedores del sistema.

El diagrama de la figura 8 ilustra los tres niveles de aislamiento del sistema. El navegador del usuario accede al puerto 8000 del host, mapeado al contenedor `clinica_web` (Django y Gunicorn). Éste se comunica con `clinica_db` (PostgreSQL 16) a través de

la red privada `clinica_net`, resolviendo el nombre del servicio `db` mediante la DNS interna de Podman. La base de datos persiste sus archivos en el volumen nombrado `pgdata`, montado en `/var/lib/postgresql/data`. **El puerto 5432 de PostgreSQL no se expone al host**: únicamente el contenedor `clinica_web` puede comunicarse con la base de datos.

5.1 Servicio web

Construye la imagen `clinica-ioo:latest` a partir del `Dockerfile`. Expone el puerto 8000 al host y se conecta a la base de datos a través del nombre de servicio `db`, resuelto automáticamente por la DNS interna de Podman.

5.2 Servicio db

Usa la imagen oficial `postgres:16-alpine`. **No** expone su puerto al host por razones de seguridad: únicamente el contenedor `web` puede comunicarse con la base a través de la red privada `clinica_net`. Implementa un *healthcheck* con `pg_isready` para que Podman conozca su estado.

5.3 Persistencia

El volumen nombrado `pgdata` se monta en `/var/lib/postgresql/data`. Esto garantiza que la información persista entre reinicios, reconstrucciones de la imagen del servicio web, y apagados del equipo. El volumen únicamente se elimina con la opción explícita `podman-compose down -v`.

5.4 Verificación de la red privada

La red `clinica_net` aísla la comunicación entre los contenedores y permite la resolución DNS interna por nombre de servicio. Al inspeccionar la red se observan los dos contenedores conectados y el rango de direcciones asignado por Podman.

Nota sobre el nombre real: `podman-compose` antepone el nombre del proyecto (la carpeta donde reside el `compose.yml`) al nombre lógico de la red declarada. Por ese motivo, la red declarada como `clinica_net` aparece en el sistema como `trabajo-ioo_clinica_net`, tal como se observa en la figura siguiente.

```
0s ~ ● ~/trabajo-100 ~ P leon
● ▶ podman network inspect trabajo-100_clinica_net
[
  {
    "name": "trabajo-100_clinica_net",
    "id": "ef7c57aaa0241f20e095edc2ab08699e16ced0e7c0ae2255c8809fbf011353c6",
    "driver": "bridge",
    "network_interface": "podman1",
    "created": "2026-04-26T12:32:31.003503266-05:00",
    "subnets": [
      {
        "subnet": "10.89.0.0/24",
        "gateway": "10.89.0.1"
      }
    ],
    "ipv6_enabled": false,
    "internal": false,
    "dns_enabled": true,
    "labels": {
      "com.docker.compose.project": "trabajo-100",
      "io.podman.compose.project": "trabajo-100"
    },
    "ipam_options": {
      "driver": "host-local"
    },
    "containers": {
      "35d0944d6456b4d45aa8d7482c3c40701207dd9dd0d0125fe53c487ae228c6b9": {
        "name": "clinica_db",
        "interfaces": {
          "eth0": {
            "subnets": [
              {
                "ipnet": "10.89.0.2/24",
                "gateway": "10.89.0.1"
              }
            ],
            "mac_address": "2e:0a:45:88:f6:af"
          }
        }
      },
      "7f0014935c8f7fd449340c6a09c8c2f61da42623b0c29728c331f6c7b04422a6": {
        "name": "clinica_web",
        "interfaces": {
          "eth0": {
            "subnets": [
              {
                "ipnet": "10.89.0.3/24",
                "gateway": "10.89.0.1"
              }
            ],
            "mac_address": "7e:5b:53:fb:11:be"
          }
        }
      }
    }
  }
]
```

Figura 9: Salida de `podman network inspect` sobre la red del proyecto.

6 Archivos clave del proyecto

6.1 Dockerfile

```

1 FROM python:3.12-slim AS runtime
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \
4     PYTHONUNBUFFERED=1 \
5     PIP_NO_CACHE_DIR=1
6
7 WORKDIR /app
8
9 RUN apt-get update \
10    && apt-get install -y --no-install-recommends curl \
11    && rm -rf /var/lib/apt/lists/*
12
13 COPY requirements.txt .
14 RUN pip install -r requirements.txt
15
16 COPY . .
17
18 RUN chmod +x /app/entrypoint.sh \
19    && mkdir -p /app/staticfiles
20
21 EXPOSE 8000
22
23 ENTRYPOINT ["/app/entrypoint.sh"]
24 CMD ["gunicorn", "core.wsgi:application", "--bind", "0.0.0.0:8000",
    "--workers", "3", "--access-logfile", "-"]

```

Listing 2: Dockerfile

Se utiliza la imagen oficial `python:3.12-slim` como base, por su balance entre tamaño y ergonomía. El `ENTRYPOINT` se encarga de preparar la base de datos antes de ejecutar el comando principal.

6.2 compose.yml

```

1 services:
2   db:
3     image: docker.io/library/postgres:16-alpine
4     container_name: clinica_db
5     restart: unless-stopped
6     environment:
7       POSTGRES_DB: ${POSTGRES_DB}
8       POSTGRES_USER: ${POSTGRES_USER}
9       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
10    volumes:
11      - pgdata:/var/lib/postgresql/data
12    healthcheck:
13      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
14      interval: 5s
15      timeout: 3s
16      retries: 10
17    networks:
18      - clinica_net
19
20   web:
21     build:

```

```
22     context: .
23     dockerfile: Dockerfile
24     image: clinica-ioo:latest
25     container_name: clinica_web
26     restart: unless-stopped
27     env_file:
28       - .env
29     ports:
30       - "8000:8000"
31     depends_on:
32       db:
33         condition: service_healthy
34     networks:
35       - clinica_net
36
37 volumes:
38   pgdata:
39
40 networks:
41   clinica_net:
```

Listing 3: compose.yml

6.3 entrypoint.sh

El `entrypoint` se ejecuta cada vez que arranca el contenedor web. Realiza cuatro pasos secuenciales:

1. **Espera activa a PostgreSQL:** intenta una conexión con `psycopg2` cada segundo, hasta 60 intentos.
2. **Migraciones:** ejecuta `python manage.py migrate --noinput` para aplicar el esquema.
3. **Archivos estáticos:** ejecuta `collectstatic` para que `WhiteNoise` pueda servirlos.
4. **Superusuario:** si están definidas las variables `DJANGO_SUPERUSER_USERNAME` y `DJANGO_SUPERUSER_PASSWORD`, crea o actualiza el superusuario de administración.

Finalmente ejecuta el CMD del Dockerfile (`gunicorn`) con `exec "$@"`, reemplazando el proceso del shell.

6.4 Variables de entorno

Variable	Descripción
SECRET_KEY	Clave de firma de Django.
DEBUG	Modo depuración (True/False).
ALLOWED_HOSTS	Lista separada por comas de hosts permitidos.
POSTGRES_DB	Nombre de la base.
POSTGRES_USER	Usuario de la base.
POSTGRES_PASSWORD	Contraseña de la base.
DATABASE_URL	URL completa leída por <code>django-environ</code> .
DJANGO_SUPERUSER_USERNAME	Usuario administrador creado automáticamente.
DJANGO_SUPERUSER_EMAIL	Correo del administrador.
DJANGO_SUPERUSER_PASSWORD	Contraseña del administrador.

Cuadro 3: Variables de entorno del proyecto

7 Proceso de despliegue

7.1 Requisitos previos

Sobre una distribución Linux (CachyOS / Arch en el caso de este equipo), instalar:

```
1 sudo pacman -S podman podman-compose
```

7.2 Pasos de despliegue

1. Clonar el repositorio:

```
1 git clone https://github.com/Chatesito/trabajo-ioo.git
2 cd trabajo-ioo
3
```

2. Crear el archivo `.env` a partir de la plantilla:

```
1 cp .env.example .env
2
```

3. Levantar los contenedores:

```
1 podman-compose up --build -d
2
```

4. Verificar:

```
1 podman ps
2 curl http://localhost:8000/pacientes/
3
```

Para detener los servicios al finalizar la demostración basta con `podman-compose down` (sin el flag `-v`, que conservaría los datos del volumen).

7.3 Evidencias del despliegue

```

$ podman-compose up --build -d
STEP 1/11: FROM python:3.12-slim AS runtime
STEP 2/11: ENV PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 PIP_DISABLE_PIP_VERSION_CHECK=1
→ Using cache 2707d302add774108b4d731751918c40ef2d6692bb42ada03610b3476468ae48
→ 2707d302add7
STEP 3/11: WORKDIR /app
→ Using cache 742e82b6a5d7f811f8728db76dad83036a126158d1ace6758b77dc1dff440354
→ 742e82b6a5d7
STEP 4/11: RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
→ Using cache c7f6c2d7c33badbf8cf028da7c3deaf32e44f59d1dbd15787765c5ac12a45f1b
→ c7f6c2d7c33b
STEP 5/11: COPY requirements.txt .
→ Using cache f6713b4d966c3836eff4d5728488669d7a0b3af44ff0df70f9f7ed1c5791d670
→ f6713b4d966c
STEP 6/11: RUN pip install -r requirements.txt
→ Using cache 3b83fa1c9db3f59cb6f6b83d77e31814b62df3293373b3896761e7a33f62d615
→ 3b83fa1c9db3
STEP 7/11: COPY . .
→ a3490f3f93fe
STEP 8/11: RUN chmod +x /app/entrypoint.sh && mkdir -p /app/staticfiles
→ 6f2a71f9a2e0
STEP 9/11: EXPOSE 8000
→ c2cb7128eab9
STEP 10/11: ENTRYPOINT ["/app/entrypoint.sh"]
→ 28abc0c8a5f2
STEP 11/11: CMD ["gunicorn", "core.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "3", "--access-logfile", "-"]
COMMIT clinica-100:latest
→ dbf6b04efa77
Successfully tagged localhost/clinica-100:latest
dbf6b04efa770f249db3c04a051b91e29fcae75fa11a0031078c4ada997760e62
d82a9b673534d7bbd0c8a756b0e76064b4699355a33a33c0154c6b406f6e6e0
35d0944d6456b4d45aa8d7482c3c40701207dd9dd0d0125fe53c487ae228c6b9
7f0814935c8f7fd449340c6a09c8c2f61da42623b0c29728c331f6c7b04422a6
clinica_db
clinica_web

```

Figura 10: Salida de `podman-compose up --build -d`: construcción de la imagen y arranque de los servicios.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
35d0944d6456	docker.io/library/postgres:16-alpine	postgres	46 seconds ago	Up 46 seconds (healthy)	5432/tcp	clinica_db
7f0814935c8f	localhost/clinica-100:latest	gunicorn core.wsgi...	46 seconds ago	Up 46 seconds	0.0.0.0:8000→8000/tcp	clinica_web

Figura 11: Contenedores en ejecución (`podman ps`). Obsérvese el estado (healthy) en `clinica_db`.

7.4 Credenciales del administrador (entorno de demostración)

Nota: las credenciales que se muestran a continuación corresponden a un entorno de demostración aislado. No deben utilizarse en ningún despliegue de producción.

El `entrypoint.sh` crea automáticamente el siguiente superusuario en el admin de Django a partir de variables de entorno:

- URL: `http://localhost:8000/admin/`
- Usuario: Leon
- Contraseña: Leon123@

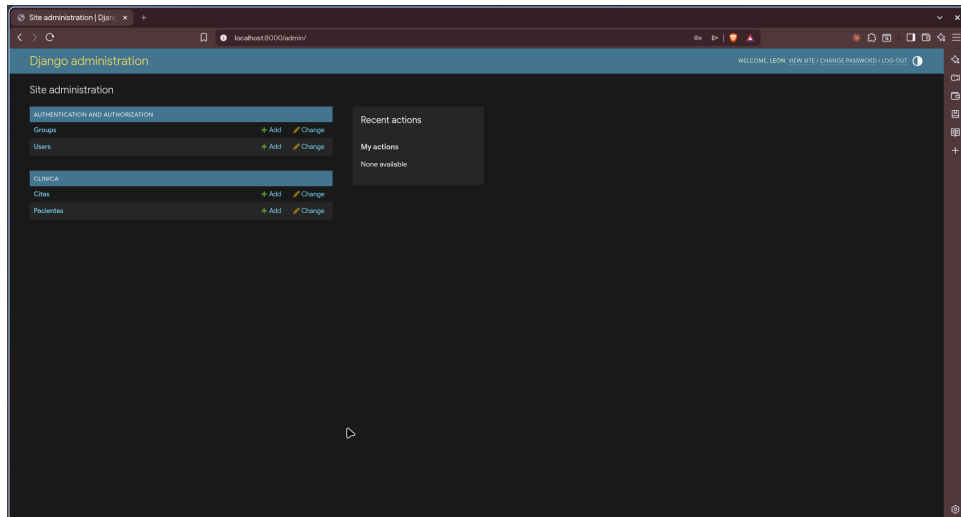


Figura 12: Panel de administración de Django con las entidades Paciente y Cita registradas.

8 Verificación de la persistencia

Para demostrar que los datos no se pierden entre ejecuciones se siguió este procedimiento:

1. Se levantaron los contenedores con `podman-compose up -d`.
2. Se crearon varios pacientes y citas desde la interfaz web.
3. Se detuvieron los contenedores con `podman-compose down` (sin el flag `-v`).
4. Se volvieron a levantar con `podman-compose up -d`.
5. Se comprobó que todos los pacientes y citas seguían disponibles.

Esto es posible gracias al volumen nombrado `pgdata`, que Podman reconoce y re-utiliza en todos los arranques posteriores al primero.

```
0s ~ ~/trabajo-100 ~ leon
• ▶ podman volume ls
DRIVER      VOLUME NAME
local      vin-tracker-pgdata
local      trabajo-100_pgdata
local      leon-inventorys_postgres_data
local      leon-inventorys_redis_data
```

Figura 13: Volumen `pgdata` listado por Podman (`podman volume ls`). Sobrevive a `podman-compose down`.

9 Publicación en Docker Hub

Para esta entrega se utilizan dos cuentas distintas, separadas por servicio:

- **Repositorio de código** (GitHub): cuenta personal de Juan David Rivera (Chatesito) — <https://github.com/Chatesito/trabajo-ioo>.
- **Imagen del contenedor** (Docker Hub): cuenta personal de Jorge León Chavarro (donleonz) — <https://hub.docker.com/r/donleonz/clinica-ioo>.

9.1 Autenticación

Desde Podman:

```
1 podman login docker.io
```

9.2 Etiquetado

```
1 podman tag clinica-ioo:latest docker.io/donleonz/clinica-ioo:latest
2 podman tag clinica-ioo:latest docker.io/donleonz/clinica-ioo:v1.0
```

9.3 Publicación

```
1 podman push docker.io/donleonz/clinica-ioo:latest
2 podman push docker.io/donleonz/clinica-ioo:v1.0
```

La imagen queda públicamente disponible en Docker Hub y cualquier persona puede descargarla y ejecutarla con:

```
1 podman pull docker.io/donleonz/clinica-ioo:latest
```

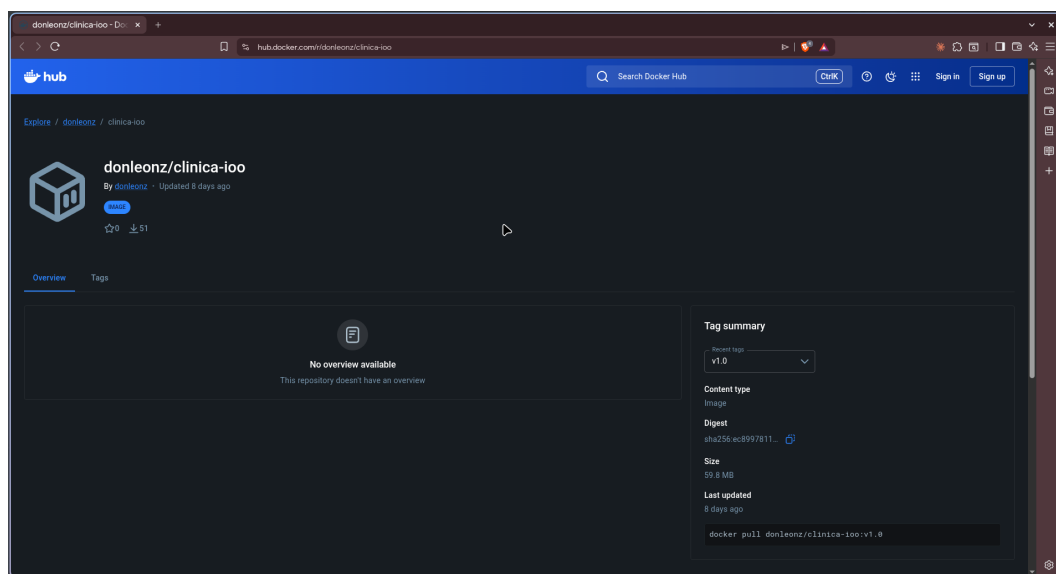


Figura 14: Repositorio público de la imagen en Docker Hub: donleonz/clinica-ioo.

10 Conclusiones

A continuación se evalúa el cumplimiento de los objetivos planteados al inicio del informe y se recogen las lecciones técnicas más relevantes del proceso.

10.1 Cumplimiento de objetivos

- **CRUD orientado a objetos con dos entidades:** cumplido. Las clases `Paciente` y `Cita` modelan el dominio mediante el ORM de Django, con validadores encapsulados, relación 1 a N por `ForeignKey` y vistas basadas en clases (*Class-Based Views*) que reutilizan jerarquías de Django.
- **Arquitectura multi-contenedor con separación de responsabilidades:** cumplido. El servicio `web` (Django + gunicorn) y el servicio `db` (PostgreSQL 16) viven en contenedores independientes, conectados por una red privada y con un único puerto expuesto al host.
- **Persistencia mediante volúmenes nombrados:** cumplido y verificado experimentalmente. El volumen `pgdata` sobrevive a `podman-compose down`, reinicios del equipo y reconstrucciones de la imagen del servicio web.
- **Automatización del arranque:** cumplido. `podman-compose` orquesta en un solo comando la construcción de la imagen, la creación de la red privada y el volumen, el arranque ordenado de los contenedores (con *healthchecks*) y la ejecución de migraciones, recolección de estáticos y creación del superusuario desde el `entrypoint`.
- **Publicación en Docker Hub:** cumplido. La imagen está disponible públicamente con dos tags (`latest` y `v1.0`) y puede ejecutarse indistintamente con Podman o Docker.

10.2 Lecciones técnicas

- Los contenedores son **efímeros por diseño**; los volúmenes son la única forma correcta de garantizar la durabilidad de los datos en un servicio con estado como una base de datos.
- Una arquitectura de dos contenedores con **red privada** y un único puerto expuesto reproduce el principio de menor privilegio: la base de datos nunca es accesible desde el exterior, sólo desde el servicio que la consume.
- La espera activa del `entrypoint` sobre la base de datos resultó imprescindible: `podman-compose` no siempre respeta la condición `service_healthy`, y la verificación adicional evitó condiciones de carrera durante el primer arranque.
- Podman se confirmó como un reemplazo directo de Docker en Linux. Al respetar el formato OCI, la imagen publicada se ejecuta indistintamente con `podman` o `docker` sin modificación alguna.
- El paradigma orientado a objetos, aplicado mediante el ORM, hace explícitas las invariantes del dominio (validadores en las clases, integridad referencial en las relaciones) sin depender de la capa de formularios.

11 Referencias

- Django Software Foundation. (2025). *Django documentation* (versión 6.0). Recuperado de <https://docs.djangoproject.com/en/6.0/>
- Red Hat, Inc. (2025). *Podman documentation*. Recuperado de <https://docs.podman.io/>
- The PostgreSQL Global Development Group. (2025). *PostgreSQL official image*. Docker Hub. Recuperado de https://hub.docker.com/_/postgres
- Whitenoise contributors. (2025). *WhiteNoise documentation*. Recuperado de <https://whitenoise.readthedocs.io/>
- Open Container Initiative. (2024). *OCI image and runtime specifications*. Recuperado de <https://opencontainers.org/>
- *Tutorial de Docker — referencia audiovisual sugerida en la guía del curso* [Video]. YouTube. Recuperado de <https://www.youtube.com/watch?v=6wbf0DetPbE>
- *Tutorial de Docker Compose — referencia audiovisual sugerida en la guía del curso* [Video]. YouTube. Recuperado de https://www.youtube.com/watch?v=CV_Uf3Dq-EU
- Repositorio del proyecto: Rivera Chaté, J. D. (2026). *trabajo-oo* [Código fuente]. GitHub. <https://github.com/Chatesito/trabajo-oo>